

ASSIGNMENT – 2

Student Record Management System using MongoDB

Student Name	Ushasri
Roll No.	A24126552120
Branch	CSM
Section	CSM-B
Topic	MongoDB
Database	collegeDB
Collection	students

Objective

To develop a simple student record management system using MongoDB to store, retrieve, update, search, sort and delete student information, and to demonstrate the usefulness of indexing.

1. Assignment Requirements

Create a MongoDB database named **collegeDB** and a collection named **students**. Each student document contains rollNo, name, branch, year, marks and email. The required operations are shown below.

1. Insert at least 5 student records.
2. Display all students.
3. Display students belonging to a particular branch.
4. Display students who scored more than 75 marks.
5. Search for a student using rollNo.
6. Search students using a condition such as marks or year.
7. Update the marks of a particular student.
8. Update another field such as email or branch.
9. Delete a student record using rollNo.
10. Display students in descending order of marks.
11. Create an index on rollNo and demonstrate why it is useful.
12. Real-time extension: above 80, below 50, highest-scoring student, branch search and sorting.

2. Student Records

Roll No.	Name	Branch	Year	Marks	Email
23CM001	Aishu	CSE-AIML	3	85	aishu@example.com
23CM002	Shiv	CSE	3	92	shiv@example.com
23CM003	Uma	ECE	3	68	uma@example.com
23CM004	Lasya	CSE-AIML	2	45	lasya@example.com
23CM005	Venky	IT	3	78	venky@example.com

The records were created in the **collegeDB.students** collection. The insert operation was performed in MongoDB before running the remaining queries.

3. Program / MongoDB Commands

The following commands implement the operations specified in the assignment. The insert block is included in the record for completeness; because the records were already inserted during execution, it should not be run again on the same collection unless the collection is cleared first.

```
use collegeDB

// 1. Insert at least 5 students
db.students.insertMany([
  {rollNo:"23CM001", name:"Aishu", branch:"CSE-AIML", year:3, marks:85, email:"aishu@example.com"},
  {rollNo:"23CM002", name:"Shiv", branch:"CSE", year:3, marks:92, email:"shiv@example.com"},
  {rollNo:"23CM003", name:"Uma", branch:"ECE", year:3, marks:68, email:"uma@example.com"},
  {rollNo:"23CM004", name:"Lasya", branch:"CSE-AIML", year:2, marks:45, email:"lasya@example.com"},
  {rollNo:"23CM005", name:"Venky", branch:"IT", year:3, marks:78, email:"venky@example.com"}
])

// 2. Display all students
db.students.find()

// 3. Particular branch
db.students.find({branch:"CSE-AIML"})

// 4. Marks more than 75
db.students.find({marks:{>75}})

// 5. Search by rollNo
db.students.find({rollNo:"23CM001"})

// 6. Search based on condition
db.students.find({year:3})

// 7. Update marks
db.students.updateOne(
  {name:"Aishu"},
  {$set:{marks:90}}
)

// 8. Update another field
db.students.updateOne(
  {name:"Shiv"},
  {$set:{email:"shivnew@example.com"}}
)

// 9. Descending order of marks
db.students.find().sort({marks:-1})

// 10. Create index on rollNo
db.students.createIndex({rollNo:1})

// 11. Display indexes
db.students.getIndexes()

// 12. Real-time: above 80
db.students.find({marks:{>80}})

// 13. Real-time: below 50
db.students.find({marks:{<50}})

// 14. Highest-scoring student
db.students.find().sort({marks:-1}).limit(1)

// 15. Particular branch
db.students.find({branch:"CSE-AIML"})

// 16. Sort according to marks
db.students.find().sort({marks:-1})

// 17. Delete using rollNo
db.students.deleteOne({rollNo:"23CM005"})

// 18. Demonstrate index usefulness
db.students.find({rollNo:"23CM001"}).explain("executionStats")
```

4. Output 1 – Display all students

Command: `db.students.find()`

Aishu	→	23CM001	CSE-AIML	Year 3	Marks 85
Shiv	→	23CM002	CSE	Year 3	Marks 92
Uma	→	23CM003	ECE	Year 3	Marks 68
Lasya	→	23CM004	CSE-AIML	Year 2	Marks 45
Venky	→	23CM005	IT	Year 3	Marks 78

5. Output 2 – Branch and marks queries

Command: `db.students.find({branch:"CSE-AIML"})` `db.students.find({marks:{$gt:75}})`

CSE-AIML:

Aishu	23CM001	85
-------	---------	----

Lasya	23CM004	45
-------	---------	----

Marks > 75:

Aishu	85
-------	----

Shiv	92
------	----

Venky	78
-------	----

6. Output 3 – Search and condition

Command: `db.students.find({rollNo:"23CM001"})` `db.students.find({year:3})`

Roll No 23CM001:

Aishu | CSE-AIML | Year 3 | Marks 85

Year 3:

Aishu | Shiv | Uma | Venky

7. Output 4 – Update operations

Command: `db.students.updateOne({name:"Aishu"},{$set:{marks:90}})`
`db.students.updateOne({name:"Shiv"},{$set:{email:"shivnew@example.com"}})`

Aishu marks updated:
matchedCount: 1
modifiedCount: 1

Shiv email updated:
matchedCount: 1
modifiedCount: 1

8. Output 5 – Descending marks

Command: `db.students.find().sort({marks:-1})`

1.	Shiv	-	92
2.	Aishu	-	90
3.	Venky	-	78
4.	Uma	-	68
5.	Lasya	-	45

9. Output 6 – Real-time extension

Command: `db.students.find({marks:{$gt:80}})` `db.students.find({marks:{$lt:50}})`
`db.students.find().sort({marks:-1}).limit(1)`

Marks above 80:

Shiv - 92

Aishu - 90

Marks below 50:

Lasya - 45

Highest-scoring student:

Shiv - 92

10. Output Evidence – MongoDB Compass

Actual execution screenshot from the MongoDB students collection showing the stored student documents.

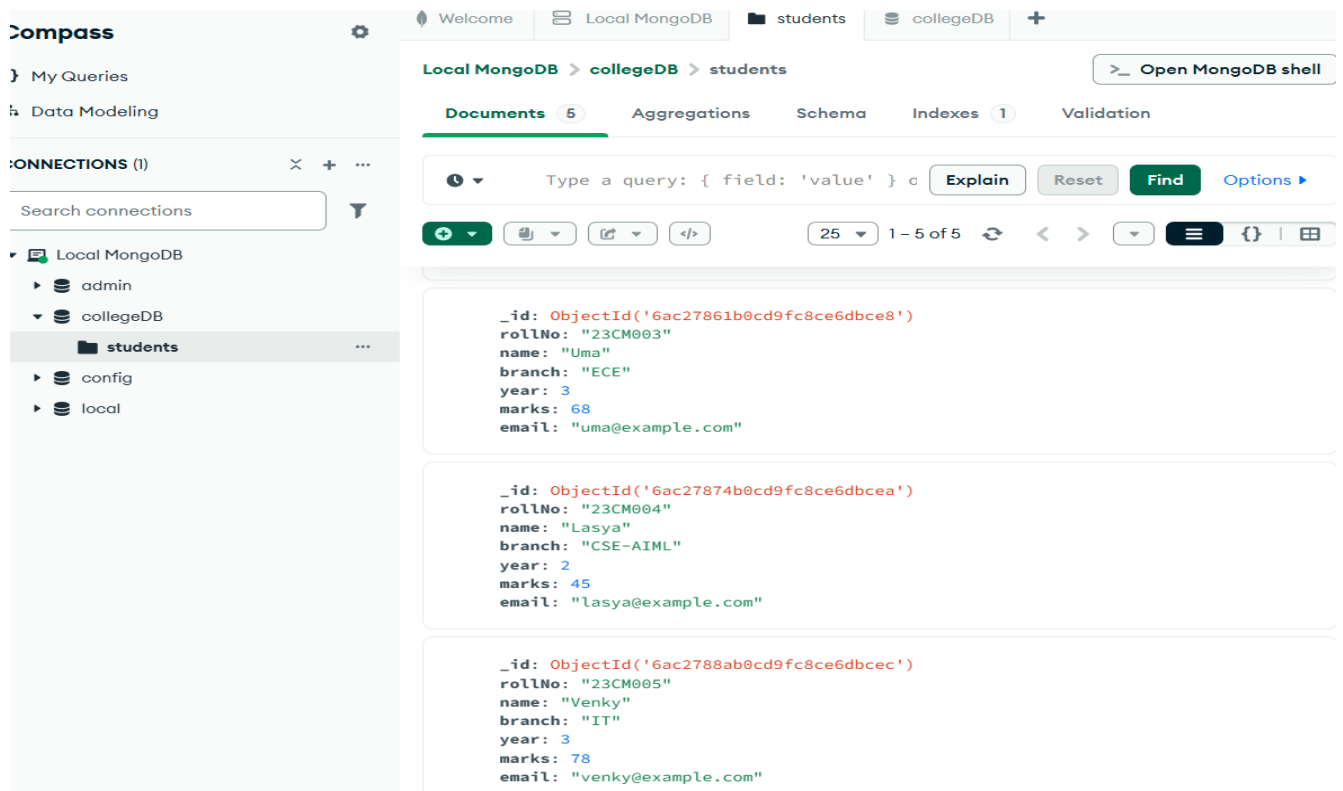
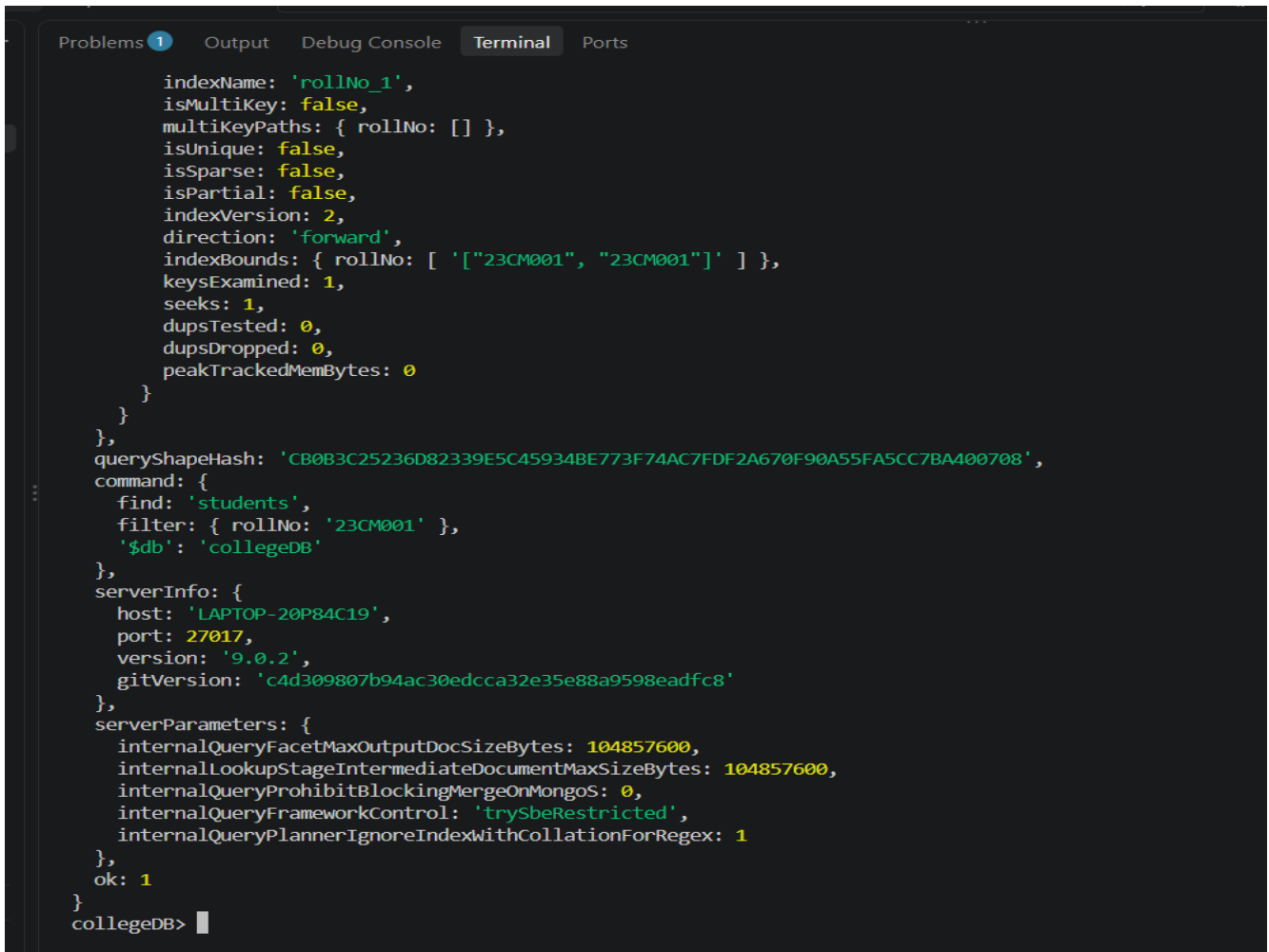


Figure 1: Students collection in MongoDB Compass.

11. Output Evidence – Indexing

The following actual terminal screenshot demonstrates that the rollNo index was used. The execution statistics show **indexName: rollNo_1**, **keysExamined: 1** and **seeks: 1**.



```
Problems 1 Output Debug Console Terminal Ports

  indexName: 'rollNo_1',
  isMultiKey: false,
  multiKeyPaths: { rollNo: [] },
  isUnique: false,
  isSparse: false,
  isPartial: false,
  indexVersion: 2,
  direction: 'forward',
  indexBounds: { rollNo: [ '['23CM001', '23CM001']' ] },
  keysExamined: 1,
  seeks: 1,
  dupsTested: 0,
  dupsDropped: 0,
  peakTrackedMemBytes: 0
}
},
queryShapeHash: 'CB0B3C25236D82339E5C45934BE773F74AC7FDF2A670F90A55FA5CC7BA400708',
command: {
  find: 'students',
  filter: { rollNo: '23CM001' },
  '$db': 'collegeDB'
},
serverInfo: {
  host: 'LAPTOP-20P84C19',
  port: 27017,
  version: '9.0.2',
  gitVersion: 'c4d309807b94ac30edcca32e35e88a9598eadfc8'
},
serverParameters: {
  internalQueryFacetMaxOutputDocSizeBytes: 104857600,
  internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
  internalQueryProhibitBlockingMergeOnMongoS: 0,
  internalQueryFrameworkControl: 'trySbeRestricted',
  internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
},
ok: 1
}
collegeDB> |
```

Figure 2: explain("executionStats") showing use of the rollNo index.

12. Output 7 – Index on rollNo

Command: db.students.getIndexes()

```
[
  {
    v: 2,
    key: { _id: 1 },
    name: "_id_"
  },
  {
    v: 2,
    key: { rollNo: 1 },
    name: "rollNo_1"
  }
]
```

Result:
The rollNo index was successfully created.

13. Indexing – Why it is useful

An index creates a faster lookup structure for a field that is frequently searched. Here, an index was created on **rollNo**. The execution statistics showed **indexName: rollNo_1**, **keysExamined: 1** and **seeks: 1**. This demonstrates that MongoDB used the rollNo index for the search instead of scanning all student documents. Therefore, indexing is useful for improving search performance, especially when the collection becomes large.

Index command:

```
db.students.createIndex({rollNo:1})
```

Verification command:

```
db.students.getIndexes()
```

14. Output 8 – Delete student record

Command: `db.students.deleteOne({rollNo:"23CM005"})`

```
{
  acknowledged: true,
  deletedCount: 1
}
```

Result:
Venky (23CM005) was deleted using rollNo.

15. Result

Thus, the Student Record Management System was successfully implemented using MongoDB. The database **collegeDB** and collection **students** were used to perform insertion, retrieval, branch and condition-based searching, updates, deletion and sorting. An index on **rollNo** was created and its usefulness was demonstrated using execution statistics. The real-time extension queries for marks above 80, below 50, highest marks, branch filtering and sorting were also performed successfully.

Conclusion

MongoDB provides a flexible document-based approach for managing student records. Queries and indexes make it convenient to retrieve and manage student information efficiently.